



Autonomous Research Agent

Building a **Plan** → **Search** → **Read** → **Synthesise** → **Review** workflow with LangGraph. An end-to-end autonomous research pipeline with real web search and real LLM calls

Today's Goal

What we're building today

An agent that takes **one topic** and autonomously researches it from start to finish, no human intervention between input and final report.



No human in the loop between input and final report.

01

Plan search angles

Break the topic into 3 specific, targeted queries

02

Search the web

Run each query through Tavily, collect URLs

03

Read full articles

Fetch complete article text from top results

04

Write a report

Synthesise gathered content into a coherent report

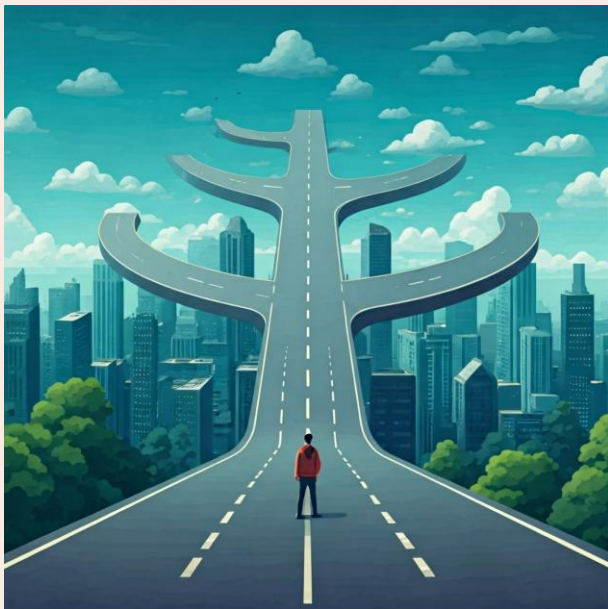
05

Review for accuracy

Self-check the report against original sources

Why Not Just One LLM Call?

The problem with asking the LLM directly



Training Data Cutoff

The LLM's knowledge has a hard cutoff date – it simply doesn't know about recent events, papers, or developments.

Shallow, Generic Answers

One giant prompt asking for everything tends to produce broad, surface-level responses that lack depth and specificity.

Specialization Wins

Breaking the job into smaller, specialized steps – each done well – produces dramatically better results than one monolithic call.

What is LangGraph?

LangGraph

A **state machine**: a shared dictionary (the state) flows through a sequence of functions (nodes).

Each node reads the state, does its job, and hands back only what it changed.

Three Building Blocks



State

The shared notebook – a dictionary every node can read from and write to



Nodes

Plain Python functions – each one reads state, does its job, returns what changed

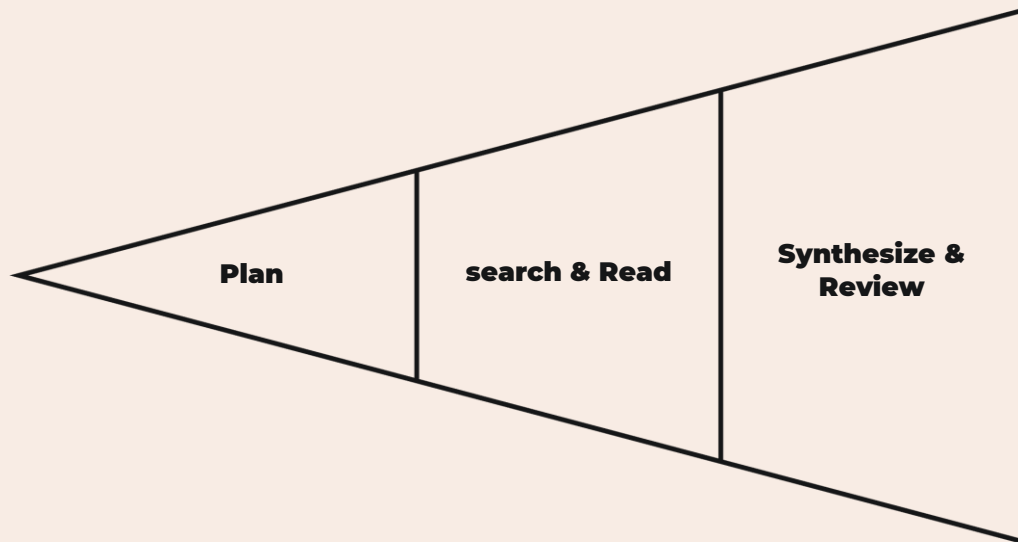


Edges

The wiring that decides what runs next – enabling loops and branching

Our Workflow, Visually

Plan → Search → Read → Synthesise → Review



Plan Turn a vague topic into 3 specific search angles	Search Run each query through Tavily, collect URLs	Read Fetch full article text from top results
Synthesise LLM writes a report from gathered content		Review LLM fact-checks its own report against sources

Why Plan First?

The thinking behind the Plan step



A raw topic like "AI in education" returns generic, broad search results, not useful for a real report.

Breaking the topic into **3 specific angles** before searching gets sharper, more useful results. This mirrors how a real researcher works: narrow the question before going looking for answers.

1

Angle 1

A focused, specific sub-question derived from the broad topic

2

Angle 2

A different dimension or perspective on the same topic

3

Angle 3

A third targeted query to ensure broad but precise coverage



Why Search AND Read Are Separate Steps

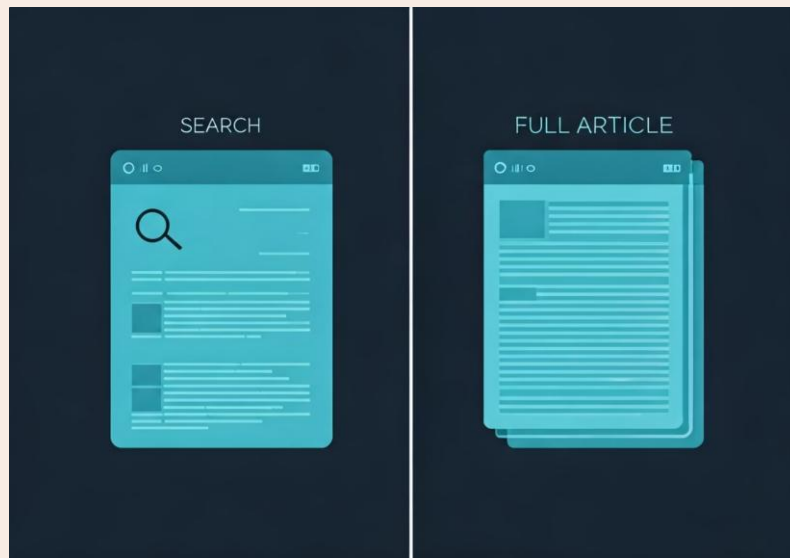
Search finds. Read understands.

Search

Returns short snippets – a sentence or two per result. Fast, but nowhere near enough content to write a real, substantive report from.

Read

Fetches the **full article text** from the most promising links. This is the actual source material the report will be grounded in.



Splitting these into two steps keeps each one **focused on a single job** – a core principle of well-designed agent workflows.



Grounding the Report

Why Synthesise is told to use **ONLY** the sources

"Only use the source text provided – not your own background knowledge."

This single instruction is what keeps the report **trustworthy**. Without it, the model may blend in outdated or simply wrong information from its training data – and the user would have no way to know.

Without Grounding

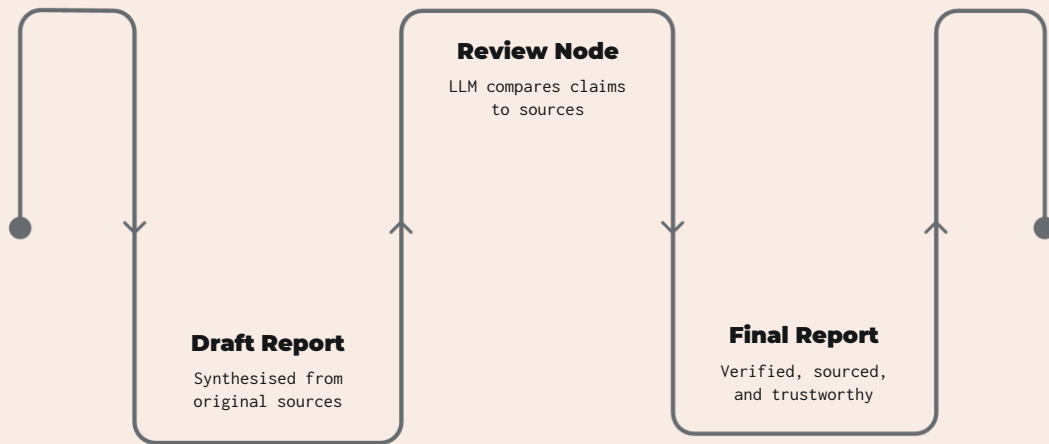
The LLM blends retrieved content with its own training data – mixing accurate sources with potentially outdated or hallucinated facts.

With Grounding

Every claim in the report traces back to a specific retrieved source. The output is verifiable and trustworthy by design.

Why We Add a Review Step

Self-checking before the user ever sees the report



The Review node receives **both the original sources and the draft report**. Its job is to find any claim not actually backed by the sources, soften or remove it, and append a source list.

i This is fact-checking – done by the model on itself, before the human ever sees the output.

What breaks if you skip a step

→ **No Plan** → vague, generic search results

→ **No Read** → report written from one-sentence snippets

→ **No Review** → hallucinated claims slip through unchecked

From Concept to Code

What you're about to see

01

Five Python Functions

One per node. Each takes the shared state dictionary in, returns only the piece it changed - nothing more.

02

A Few Lines of Wiring

Connect the five nodes together in order using LangGraph's edge API.

03

One Line to Run

Invoke the compiled graph end-to-end with a single topic string.

- Today's scope: a **straight-line 5-step pipeline** with real web search and real LLM calls.

Real Web Search

Tavily API - live results,
not mocked data

Real LLM Calls

Actual model inference at
Plan, Synthesise, and
Review nodes

Let's look at the code. 🚀